

```

from google.colab import drive
drive.mount('/content/drive')

import polars as pl

file_path =
'/content/drive/MyDrive/Amazon_Musical/Musical_Instruments.jsonl'

try:
    df_polars = pl.read_ndjson(file_path)
    print(df_polars.head())
except Exception as e:
    print(f"An error occurred: {e}")

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

shape: (5, 10)

rating	title	text	images	...	user_id	
timestamp	helpful_v	verified_				
---	---	---	---		---	---
ote	purchase					
f64	str	str	list[stru		str	i64
---	---		ct[4]]			
i64	bool					
5.0	Five Stars	Great head	[]	...	AGKASBHYZ	
145265058	0	true phones,			PGTEP06LW	6000
		comfortabl			ZPVJWB2BV	
		e ...			A	
3.0	nice	I like the	[]	...	AGCI7FAH4	
155856736	2	true			GL5FI65HY	5290
	sound.	piano..			LKWTMFZ2C	
	pedal	but the			Q	
	failed	sus...				
	afte...					
4.0	okay	pretty	[]	...	AGCI7FAH4	
138491248	0	true			GL5FI65HY	2000
		good				

		overall.		LKWTMFZ2C	
		I like i...		Q	
3.0	Easy to	Too bad it	[]	...	AEM663T6X
160705569	0	true			
	return.	didn't		HZFWL0DF4	3671
		work. At		US2RC0CUS	
		lea...		A	
5.0	Good	Good and	[]	...	AFJTRBXMU
162259578	0	false			
	product	sturdy but		RLHS5EGNX	5255
	despite	the bolt		LUHDHIZRF	
	tight bol...	w...		Q	

```
# Mount Google Drive (Colab)
# from google.colab import drive
# drive.mount('/content/drive')

import time
from pyspark.sql import SparkSession

# ---- Spark session ----
spark = (SparkSession.builder
        .appName("Read_JSONL_from_Drive")
        .getOrCreate())

# ---- File path (Drive is usually /content/drive in Colab) ----
file_path =
"/content/drive/MyDrive/Amazon_Musical/Musical_Instruments.jsonl"

# ---- Time execution ----
start = time.time()

# JSONL (newline-delimited JSON): Spark reads it with read.json
df = spark.read.json(file_path)

elapsed = time.time() - start
```


[illegible]

|B07T9NM5QR |5.0 |Good and sturdy but the bolt was hell to get out. I thought it was glued in. I had to get a hammer and whack it out of place. Overall, I'm satisfied.
|1622595785255|Good product despite tight bolt.
|AFJTRBXMURLHS5EGNXLUHDHIZRFQ|false |
|B00A2DBHMA|0 |[]
|B00A2DBHMA |5.0 |works great with my tablet
|1546556517459|bluetooth key board
|AEFKF6R2GUSK2AWPSWRR4Z036JVQ|true |
|B07Z9M4RS1|1 |[]
|B07Z9M4RS1 |5.0 |Xlnt quality. Does NOT work for thick music books. Xlnt for half inch thick music books.
|1586394792148|Works great for thinner books.
|AFWHJ603PV4JC7PVOJH6CPUL02KQ|true |
|B07K8FQFN8|0 |[]
|B0C5D8X3JT |5.0 |Easy to use. Great quality!
|1586394628934|Xlnt product
|AFWHJ603PV4JC7PVOJH6CPUL02KQ|true |
|B01EVKUHE5|3 |[]
|B074VB2D2K |5.0 |My granddaughter loved. I think the adults had fun with it too. Sound was great!
|1482444129000|Nice Karaoke machine
|AH6CATODIVPVUOJEWHRSCSKA0HA|true |
|B0037JT95I|0 |[]
|B0037JT95I |5.0 |The Singing Machine SML-385 is a cute, fun, easy to use machine. My family uses it all the time. So easy to use even my 6 year old grandson loves it!
|1428847129000|fun, easy to use machine
|AGJSTXSEWFVQWVQIN06QKMNBUNNA|true |
|B001SGWKR6|0 |[]
|B001SGWKR6 |5.0 |OMG, the best! My grandson just loves his new keyboard, he likes the ease of use and how it is compatible with his laptop,
|1428846808000|the best! My grandson just loves his new keyboard
|AGJSTXSEWFVQWVQIN06QKMNBUNNA|true |
|B07MGVQ5KH|0 |[]
|B07MGVQ5KH |5.0 |This is a super cool product. I live in a new 'cool' 55 and older community in the Daytona Beach area and we have parties at neighbor homes several times a week. When I first set these up I wasn't sure what to expect. But after seeing the variety of lights, patterns and speeds I knew this would be a hit. We had a Motown party last weekend and this projector was the highlight of the night! Great colors, great remote control capability and even responds to voice commands. This would be a great addition to any home (young or old) for use when there is a party to be had. I definitely highly recommend this gadget for all ages and holidays!|1565630767134|Super projecting lights for all party uses! Amazingly good!|
AFSKPY37N3C43S0I5IEXEK5JSIYA|false |
|B07GYXJZRY|1 |[]

|B0BXCXGCVN |5.0 |My son is a pilot in the Air Force and uses these daily when walking around the runways. He states they are super comfortable and sometimes doesn't even realize he has them in his ears. They help block the transmission noises from the jets and protect his hearing. Although you might be able to purchase higher quality ones, these are great for the price and really do work. I also have a pair which I have used when attending concerts and it also helps protect me when I am close to the stage speakers minimizing the high decibels but still allowing me to hear the concert overall.
|1555324663865|Great reasonably priced way to protect your hearing
|AFSKPY37N3C43S0I5IEXEK5JSIYA|false |
|B074R9F26D|0 |[]
|B074R9F26D |5.0 |Gave to son-in-law in his xmas stocking... he obviously is a musician. He smiled and thanked me nicely... guitar picks are very personal... As he pointed out, he goes through many. So this is a nice collection.
|1516941193158|guitar pick collections
|AFZUK3MTBIBEDQOPAK30ATUOUKLA|true |
|B002IRID0C|0 |[]
|B074MK32SL |5.0 |Husband ordered for packing his expensive devices used on jobs. Says this box is super tough and does the job. It's not real pretty, but function is the point. Arrived in quickly (we have Prime) and in good condition
|1410987748000|works great.
|AFZUK3MTBIBEDQOPAK30ATUOUKLA|true |
|B000FET3DG|0 |[]
|B000FET3DG |5.0 |My 3 year old grandson loves this harmonia. Daddy has one and he is learning to play. It's a good size and honestly I bought it because my maiden name is Johnson. :) Has a great hard case and a cleaning cloth.
|1397779929000|grandson loves it
|AFZUK3MTBIBEDQOPAK30ATUOUKLA|true |
|B07BHLGK87|0 |[]
|B09W5LPHBZ |5.0 |Great quality! We got this for our daughters birthday and she absolutely loves it! It came in an awesome case and a product satisfaction guarantee.
|1549315341122|My daughter loved it!
|AGFUJ3EGB7VTLV3ZMKEZVRJHPA5Q|true |
|B002SCZR8X|0 |[]
|B07DLCDT3N |2.0 |This product is very light I'm afraid if I drop it on any surface other than carpet it will bend or chip. I am unable to blow a note but I've never used this instrument so it will take much practice.
|1555608996628|Flute like
|AHFURURV4H35BXUUTXYMJYA36CTQ|true |
|B01MUAMT4T|4 |[{IMAGE, <https://images-na.ssl-images-amazon.com/images/I/71-Nm7DAvtL.jpg>, https://images-na.ssl-images-amazon.com/images/I/71-Nm7DAvtL._SL800_.jpg, https://images-na.ssl-images-amazon.com/images/I/71-Nm7DAvtL._SL256_.jpg}, {IMAGE,

[illegible]

+-----+-----+

only showing top 20 rows

```
DataFrame[asin: string, helpful_vote: bigint, images:
array<struct<attachment_type:string,large_image_url:string,medium_imag
e_url:string,small_image_url:string>>, parent_asin: string, rating:
double, text: string, timestamp: bigint, title: string, user_id:
string, verified_purchase: boolean]
```

Execution time (read + basic actions): 42.511 seconds

```
!pip -q install bertopic sentence-transformers umap-learn hdbscan
scikit-learn panda
```

Preparing metadata (setup.py) ...

154.7/154.7 kB 7.1 MB/s eta
0:00:00

```
from google.colab import drive
drive.mount('/content/drive')
```

```
import polars as pl
```

```
file_path =
'/content/drive/MyDrive/Amazon_Musical/Musical_Instruments.jsonl'
```

Pre-processing

```
import re
import time
import pandas as pd
```

```
from google.colab import drive, files
drive.mount('/content/drive')
```

```
# ---- Input / Output ----
```

```
file_path =
"/content/drive/MyDrive/Amazon_Musical/Musical_Instruments.jsonl"
out_csv_path = "/content/Amazon_low_ratings_cleaned.csv" # saved to
Colab VM for download
```

```
# ---- 1) Text preprocessing function ----
```

```
# Goal: normalize text for topic modeling (BERTopic)
```

```
url_re = re.compile(r"https?:\/\/\S+|www\.\S+")
```

```
html_re = re.compile(r"<.*?>")
```

```
non_alnum_re = re.compile(r"[^a-z0-9\s]") # keep
letters/numbers/spaces
```



```

multi_space_re = re.compile(r"\s+")

def clean_text(s: str) -> str:
    if not isinstance(s, str):
        return ""
    s = s.lower()
    s = html_re.sub(" ", s)
    s = url_re.sub(" ", s)
    s = s.replace("\n", " ").replace("\r", " ").replace("\t", " ")
    s = non_alnum_re.sub(" ", s)
    s = multi_space_re.sub(" ", s).strip()
    return s

# ---- 2) Read JSONL in chunks, filter rating in {1,2,3}, select
# columns, clean ----
start = time.time()

chunks = []
chunk_size = 200_000 # adjust based on RAM
wanted_cols = ["rating", "text", "parent_asin"]

for chunk in pd.read_json(file_path, lines=True,
    chunksize=chunk_size):
    # keep only the needed columns (if present)
    existing = [c for c in wanted_cols if c in chunk.columns]
    chunk = chunk[existing].copy()

    # ensure rating numeric
    chunk["rating"] = pd.to_numeric(chunk["rating"], errors="coerce")

    # ---- Select rows with rating 1,2,3 ----
    chunk = chunk[chunk["rating"].isin([1, 2, 3])]

    # fill missing text, clean, and drop empty after cleaning
    chunk["text"] =
    chunk["text"].fillna("").astype(str).map(clean_text)
    chunk = chunk[chunk["text"].str.len() > 0]

    # parent_asin cleanup (optional)
    if "parent_asin" in chunk.columns:
        chunk["parent_asin"] =
        chunk["parent_asin"].fillna("").astype(str).str.strip()

    if len(chunk) > 0:
        chunks.append(chunk)

df_clean = pd.concat(chunks, ignore_index=True) if chunks else
pd.DataFrame(columns=wanted_cols)

elapsed = time.time() - start

```

```

print(f"Rows kept: {len(df_clean):,}")
print(f"Time (read + filter + clean): {elapsed:.2f} sec")
df_clean.head()

Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
Rows kept: 592,214
Time (read + filter + clean): 59.52 sec

{"type": "dataframe", "variable_name": "df_clean"}

out_csv_path = "/content/Amazon_low_ratings_cleaned.csv"

df_clean.to_csv(out_csv_path, index=False)
print("Saved:", out_csv_path)

# Download to your local machine
files.download(out_csv_path)

Saved: /content/Amazon_low_ratings_cleaned.csv

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>


import time
import numpy as np
import pandas as pd

from bertopic import BERTopic
from sentence_transformers import SentenceTransformer
from umap import UMAP
import hdbscan
from sklearn.feature_extraction.text import CountVectorizer

# df_clean from your previous step has: rating, text, parent_asin
print("Total docs:", len(df_clean))

# ---- Sample for fitting ----
FIT_N = 150_000
df_fit = df_clean.sample(FIT_N,
random_state=42).reset_index(drop=True)
docs_fit = df_fit["text"].tolist()

# ---- Embeddings on GPU ----
embedder = SentenceTransformer("all-MiniLM-L6-v2", device="cuda")

t0 = time.time()
emb_fit = embedder.encode(
    docs_fit,

```

```

        batch_size=32,
        show_progress_bar=True,
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype(np.float16) # save RAM
    print(f"Embedding fit sample time: {time.time()-t0:.2f} sec |
    shape={emb_fit.shape}")

# ---- Models ----
umap_model = UMAP(
    n_neighbors=15,
    n_components=5,
    min_dist=0.0,
    metric="cosine",
    random_state=42
)

hdbscan_model = hdbscan.HDBSCAN(
    min_cluster_size=80, # slightly larger for stability at
    this scale
    min_samples=10,
    metric="euclidean",
    cluster_selection_method="eom",
    prediction_data=False
)

vectorizer_model = CountVectorizer(
    stop_words="english",
    ngram_range=(1, 2),
    min_df=15,
    max_features=50_000
)

topic_model = BERTopic(
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
    vectorizer_model=vectorizer_model,
    calculate_probabilities=False,
    verbose=True
)

t1 = time.time()
topics_fit, _ = topic_model.fit_transform(docs_fit,
    embeddings=emb_fit)
print(f"BERTopic fit time (sample): {time.time()-t1:.2f} sec")

topic_info = topic_model.get_topic_info()
topic_info.head(20)

```

```
/usr/local/lib/python3.12/dist-packages/hdbscan/  
robust_single_linkage.py:175: SyntaxWarning: invalid escape sequence  
'\{'  
$max \{ core_k(a), core_k(b), 1/\alpha d(a,b) \}$.
```

Total docs: 592214

```
{"model_id": "f590b46515ab473bb18547baa2897fbb", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c035ea1a7c21416a91d8ad892f9b3a3c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "96b3548d6f8a409aa5c59e790970a350", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "8d1dbb78da404a2a8491864fd8a7396c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "ae6573f18d634978b40b33677b64e4b3", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1072b1e1789c41129a2cac5d99381278", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c8bd9fc0f01142e0ab92dc4cf69f2b04", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e53594455bbb49d181f6a4131f0bdf39", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "cb495d3aa99a407390261901225746cb", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "38caa924f8df49b5966c0e70b219dbf7", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "54e0c1376148496f9ad3213834d35259", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9b055d79d6d9466bbc82ecf2e2f04e6d", "version_major": 2, "version_minor": 0}
```

2026-01-26 12:16:45,274 - BERTopic - Dimensionality - Fitting the dimensionality reduction algorithm

Embedding fit sample time: 115.90 sec | shape=(150000, 384)

2026-01-26 12:21:26,155 - BERTopic - Dimensionality - Completed ✓

2026-01-26 12:21:26,159 - BERTopic - Cluster - Start clustering the reduced embeddings

2026-01-26 12:21:39,558 - BERTopic - Cluster - Completed ✓

2026-01-26 12:21:39,588 - BERTopic - Representation - Fine-tuning

topics using representation models.

2026-01-26 12:21:54,920 - BERTopic - Representation - Completed ✓

BERTopic fit time (sample): 312.55 sec

```
{ "summary": "{\n  \"name\": \"topic_info\",\n  \"rows\": 256,\n  \"fields\": [\n    {\n      \"column\": \"Topic\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 74,\n        \"min\": -1,\n        \"max\": 254,\n        \"num_unique_values\": 256,\n        \"samples\": [\n          227,\n          5,\n          78\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 3479,\n        \"min\": 80,\n        \"max\": 55341,\n        \"num_unique_values\": 193,\n        \"samples\": [\n          460,\n          159,\n          314\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 256,\n        \"samples\": [\n          \"227_material_cheap material_cute_materials\",\n          \"5_strap_leather_straps_guitar_strap\",\n          \"78_item_product_broken_returned\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Representation\",\n      \"properties\": {\n        \"dtype\": \"object\",\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Representative_Docs\",\n      \"properties\": {\n        \"dtype\": \"object\",\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\",\n  \"variable_name\": \"topic_info\" }
```

In BERTopic, the overall pipeline is:

Embed text → semantic vectors (sentence-transformers)

UMAP → reduce dimension to help clustering

HDBSCAN → produce clusters (topics) + outliers (-1)

c-TF-IDF → extract interpretable top words per cluster

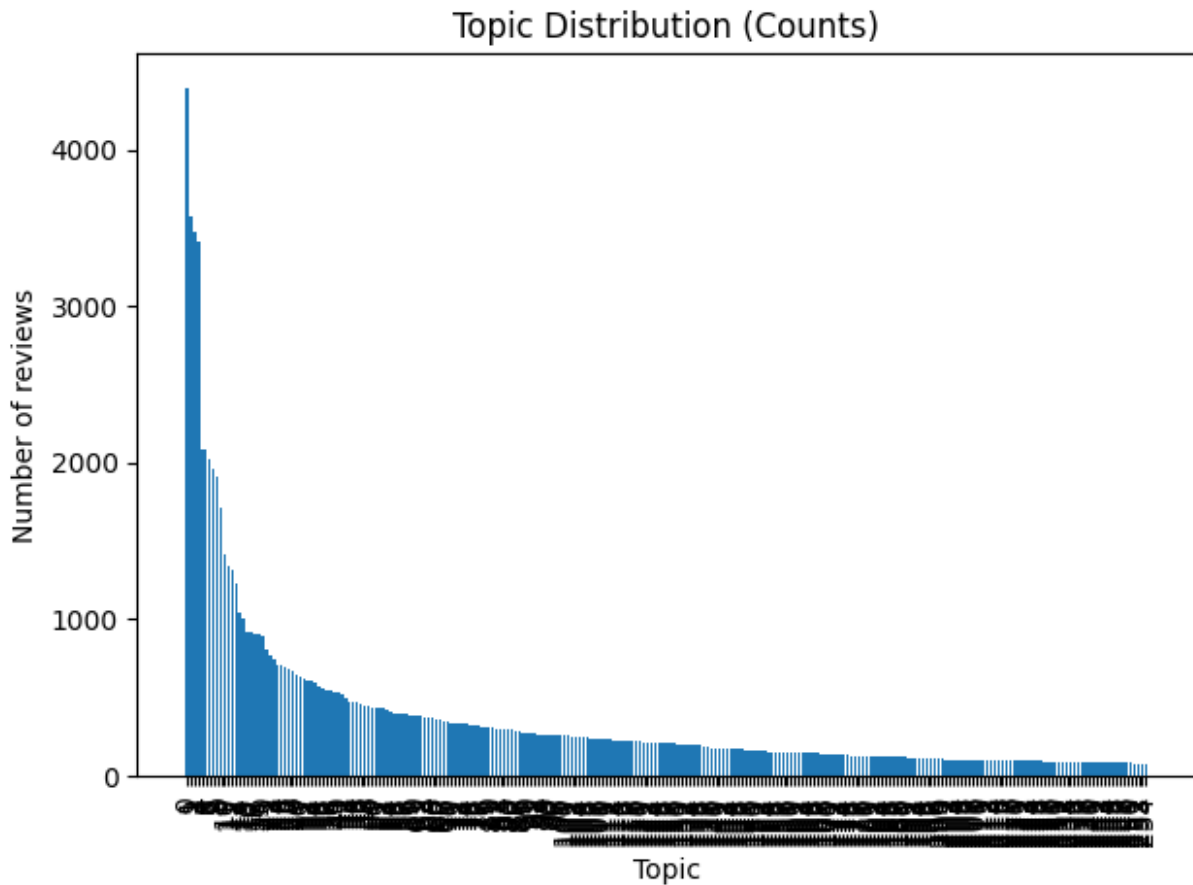
```
import matplotlib.pyplot as plt

topic_info = topic_model.get_topic_info()

# Remove outlier topic -1 if you want
topic_info_plot = topic_info[topic_info["Topic"] != -1].copy()

plt.figure()
```

```
plt.bar(topic_info_plot["Topic"].astype(str),
        topic_info_plot["Count"])
plt.title("Topic Distribution (Counts)")
plt.xlabel("Topic")
plt.ylabel("Number of reviews")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



```
fig = topic_model.visualize_barchart(top_n_topics=15)
fig.show()
```

```
t1 = time.time()
topics_fit, _ = topic_model.fit_transform(docs_fit,
                                           embeddings=emb_fit)
print(f"BERTopic fit time: {time.time()-t1:.2f}s")
```

---- 5) Transform ALL docs in batches to get topic per row ----

```

BATCH = 50_000 # reduce to 20_000 if RAM issues
all_topics = []

t2 = time.time()
for i in range(0, len(docs_all), BATCH):
    batch_docs = docs_all[i:i+BATCH]
    emb = embedder.encode(
        batch_docs, batch_size=64, show_progress_bar=True,
        convert_to_numpy=True, normalize_embeddings=True
    ).astype(np.float16)

    batch_topics, _ = topic_model.transform(batch_docs,
        embeddings=emb)
    all_topics.extend(batch_topics)

    print(f"Processed {min(i+BATCH, len(docs_all))}/{len(docs_all)}")

print(f"Total transform time: {time.time()-t2:.2f}s")

# ---- 6) Attach to df_clean ----
df_clean = df_clean.reset_index(drop=True)
df_clean["topic"] = all_topics

df_clean[["rating", "parent_asin", "text", "topic"]].head()

2026-01-26 13:16:02,301 - BERTopic - Dimensionality - Fitting the
dimensionality reduction algorithm
2026-01-26 13:20:18,543 - BERTopic - Dimensionality - Completed ✓
2026-01-26 13:20:18,547 - BERTopic - Cluster - Start clustering the
reduced embeddings
2026-01-26 13:20:31,851 - BERTopic - Cluster - Completed ✓
2026-01-26 13:20:31,879 - BERTopic - Representation - Fine-tuning
topics using representation models.
2026-01-26 13:20:47,264 - BERTopic - Representation - Completed ✓

BERTopic fit time: 288.00s

```

```

-----
-----
NameError                                Traceback (most recent call
last)
/tmp/ipython-input-3145256209.py in <cell line: 0>()
      8
      9 t2 = time.time()
--> 10 for i in range(0, len(docs_all), BATCH):
     11     batch_docs = docs_all[i:i+BATCH]
     12     emb = embedder.encode(

NameError: name 'docs_all' is not defined

```

```

import pandas as pd
import matplotlib.pyplot as plt
from collections import defaultdict

# 1-star subset (exclude outliers topic=-1 if you prefer)
df1 = df_clean[df_clean["rating"] == 1].copy()

topic_counts = df1["topic"].value_counts()

# Aggregate "word importance among 1-star reviews"
# (topic prevalence * topic word weight)
word_score = defaultdict(float)

for topic_id, cnt in topic_counts.items():
    if topic_id == -1:
        continue
    words = topic_model.get_topic(topic_id) # list of (word, weight)
    if not words:
        continue
    for w, wt in words:
        word_score[w] += float(cnt) * float(wt)

top10 = sorted(word_score.items(), key=lambda x: x[1], reverse=True)
[:10]
top_words = [w for w, _ in top10]
top_vals = [v for _, v in top10]

plt.figure()
plt.bar(top_words, top_vals)
plt.title("Top 10 Topic Words Discussed in 1-Star Reviews (Weighted by Topic Prevalence)")
plt.xlabel("Topic Word / Phrase")
plt.ylabel("Weighted Importance (topic_count × c-TF-IDF weight)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

print(f"Number of 1-star reviews: {len(df1):,}")

```

```

-----
-----
KeyError                                Traceback (most recent call
last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in
get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

```



```
index.pyx in pandas._libs.index.IndexEngine.get_loc()
```

```
index.pyx in pandas._libs.index.IndexEngine.get_loc()
```

```
pandas/_libs/hashtable_class_helper.pxi in  
pandas._libs.hashtable.PyObjectHashTable.get_item()
```

```
pandas/_libs/hashtable_class_helper.pxi in  
pandas._libs.hashtable.PyObjectHashTable.get_item()
```

```
KeyError: 'topic'
```

The above exception was the direct cause of the following exception:

```
KeyError                                Traceback (most recent call  
last)
```

```
/tmp/ipython-input-840204156.py in <cell line: 0>()  
      6 df1 = df_clean[df_clean["rating"] == 1].copy()  
      7
```

```
----> 8 topic_counts = df1["topic"].value_counts()  
      9
```

```
     10 # Aggregate "word importance among 1-star reviews"
```

```
/usr/local/lib/python3.12/dist-packages/pandas/core/frame.py in  
__getitem__(self, key)
```

```
    4100         if self.columns.nlevels > 1:  
    4101             return self._getitem_multilevel(key)  
-> 4102         indexer = self.columns.get_loc(key)  
    4103         if is_integer(indexer):  
    4104             indexer = [indexer]
```

```
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in  
get_loc(self, key)
```

```
    3810         ):  
    3811             raise InvalidIndexError(key)  
-> 3812         raise KeyError(key) from err  
    3813     except TypeError:  
    3814         # If we have a listlike key, _check_indexing_error  
will raise
```

```
KeyError: 'topic'
```

Task

Load and display the `Musical_Instruments.jsonl` file and another JSONL file from the "Amazon_Musicsl" folder in your Google Drive using PySpark.

Mount Google Drive

Subtask:

Mount Google Drive to allow PySpark to access the files stored in your 'Amazon_Musicsl' folder.

Reasoning: To mount Google Drive, I need to import the `drive` module from `google.colab` and then call the `mount()` method with the specified path.

```
from google.colab import drive
drive.mount('/content/gdrive')
```

Drive already mounted at `/content/gdrive`; to attempt to forcibly remount, call `drive.mount("/content/gdrive", force_remount=True)`.

Load Musical_Instruments.jsonl

Subtask:

Load the 'Musical_Instruments.jsonl' file into a PySpark DataFrame from the specified Google Drive path.

Reasoning: To load the 'Musical_Instruments.jsonl' file into a PySpark DataFrame, I need to first initialize a `SparkSession`, then define the file path, and finally use `spark.read.json()` to load the data.

Task

Load and display `Musical_Instruments.jsonl` and another JSONL file from the `Amazon_Musicsl` folder in your Google Drive using PySpark.

Load Musical_Instruments.jsonl

Subtask:

Load the 'Musical_Instruments.jsonl' file into a PySpark DataFrame from the specified Google Drive path.

Reasoning: To load the 'Musical_Instruments.jsonl' file into a PySpark DataFrame, I will first import `SparkSession`, then create a `SparkSession` instance, and finally use `spark.read.json()` with the provided `file_path` to load the data into a DataFrame named `df_musical_instruments`.

```

from pyspark.sql import SparkSession

# Create a SparkSession instance
spark =
SparkSession.builder.appName("MusicalInstrumentsData").getOrCreate()

# Load the JSONL file into a PySpark DataFrame
df_musical_instruments = spark.read.json(file_path)

print("DataFrame 'df_musical_instruments' created successfully.")
-----
-----
AnalysisException                                Traceback (most recent call
last)
/tmp/ipython-input-223399343.py in <cell line: 0>()
      5
      6 # Load the JSONL file into a PySpark DataFrame
----> 7 df_musical_instruments = spark.read.json(file_path)
      8
      9 print("DataFrame 'df_musical_instruments' created
successfully.")

/usr/local/lib/python3.12/dist-packages/pyspark/sql/readwriter.py in
json(self, path, schema, primitivesAsString, prefersDecimal,
allowComments, allowUnquotedFieldNames, allowSingleQuotes,
allowNumericLeadingZero, allowBackslashEscapingAnyCharacter, mode,
columnNameOfCorruptRecord, dateFormat, timestampFormat, multiLine,
allowUnquotedControlChars, lineSep, samplingRatio, dropFieldIfAllNull,
encoding, locale, pathGlobFilter, recursiveFileLookup, modifiedBefore,
modifiedAfter, allowNonNumericNumbers, useUnsafeRow)
    466         if type(path) == list:
    467             assert self._spark._sc._jvm is not None
--> 468             return
self._df(self._jreader.json(self._spark._sc._jvm.PythonUtils.toSeq(pat
h)))
    469
    470         if not is_remote_only():

/usr/local/lib/python3.12/dist-packages/py4j/java_gateway.py in
__call__(self, *args)
    1360
    1361         answer = self.gateway_client.send_command(command)
-> 1362         return_value = get_return_value(
    1363             answer, self.gateway_client, self.target_id,
self.name)
    1364

/usr/local/lib/python3.12/dist-packages/pyspark/errors/exceptions/
captured.py in deco(*a, **kw)

```

```

    286                # Hide where the exception came from that
shows a non-Pythonic
    287                # JVM exception message.
--> 288                raise converted from None
    289            else:
    290                raise

```

```

AnalysisException: [PATH_NOT_FOUND] Path does not exist:
file:/content/content/gdrive/MyDrive/Amazon_Musical/Musical_Instrument
s.jsonl. SQLSTATE: 42K03

```

```

from pyspark.sql.functions import col
from pyspark.sql import SparkSession

```

```

spark = SparkSession.builder.master("local[*]").getOrCreate()
spark.conf.set("spark.sql.repl.eagerEval.enabled", True)

```

```

f_path='/content/gdrive/MyDrive/Amazon_Musical/
Musical_Instruments.jsonl'

```

```

df_rev = spark.read.jsonl(f_path, header=True, sep=";")

```

```

-----
-----
AttributeError                                Traceback (most recent call
last)
/tmp/ipython-input-3906987829.py in <cell line: 0>()
----> 1 df_rev = spark.read.jsonl(f_path, header=True, sep=";")

AttributeError: 'DataFrameReader' object has no attribute 'jsonl'

```